minicut Video Editor

Embedded LuaJIT & Plugin API Reference Manual

LuaJIT 5.1 Runtime Dockable Tool Panels Tool Shelf Dock Network Library (libcurl) FreeType Text Generation

1. Introduction & Architecture

minicut features a fully embedded, high-performance **LuaJIT** scripting engine. minicut enables complete automation, timeline editing, clip keyframing, and custom interactive dockable tool panels.

Script Directory Locations

Scripts and icons are discovered in the user configuration directory:

Location	Trigger / Role	Description
`~/.config/minicut/autorun/`	Startup / Auto-register	Scripts execute on startup and generate an icon button on the horizontal Tool Shelf Dock below the main menu bar.
`~/.config/minicut/scripts/`	Menu Click / On Demand	Scripts appear in the top-level **Scripts** menu on the main menu bar. Clicking runs the script and opens its tool panel.
`~/.config/minicut/icons/`	Dock Icon Asset	PNG icon images matching script stems (e.g.,

Location	Trigger / Role	Description
		`keyframe_animator.png` for `keyframe_animator.lua`).

2. Plugin Script Architecture

Every minicut script defines timeline and clip manipulation callbacks, followed by a `plugin:init_panel()` method that registers Dear ImGui UI controls.

```
-- ========================================================================

-- Example Base Script: my_tool.lua

-- ========================================================================



local target_track = 0

local clip_scale = 1.25



function execute_placement()

    minicut.push_undo()

    local path = "/path/to/media.mp4"

    local clip_id = timeline.place_clip(path, target_track, timeline.get_playhead(),
5.0)

    if clip_id and clip_id ~= "" then

        clip.set_scale(clip_id, clip_scale)

        timeline.add_keyframe(clip_id, "opacity", 0.0, 0.0)

        timeline.add_keyframe(clip_id, "opacity", 1.0, 1.0)

        minicut.set_status("Placed media clip on track " .. target_track)

    end

end
```

```
function plugin:init_panel()
    gui.text("Media Placement Tool")
    gui.separator()
    target_track = gui.slider_int("Track Lane", target_track, 0, 4)
    clip_scale = gui.slider_float("Initial Scale", clip_scale, 0.1, 5.0)

    gui.button("Place Clip At Playhead", execute_placement)
end
```

3. Core Lua Modules

3.1. Project & File Automation (`project` / Global)

Function	Returns	Description
`project_save([path])` `project.save([path])`	`boolean, string`	Saves current project state to a `.minicut` JSON file.
`project_load(path)` `project.load(path)`	`boolean, string`	Loads a project from `path`, refreshing timeline and media.
`project.get_name()`	`string`	Returns the current project name.
`project.set_name(name)`	`void`	Sets the current project name.

3.2. Network & Internet Library (`net`)

Function	Returns	Description
`net.get(url)`	`table`	HTTP GET request. Returns `{status, body, ok, headers}`.

Function	Returns	Description
`net.post(url, body)`	`table`	HTTP POST request with string or JSON payload.
`net.download(url, dest)`	`boolean`	Downloads remote asset directly to local disk.
`net.upload(url, path)`	`table`	HTTP POST/upload local file binary stream.

3.3. Timeline & Track Management (`timeline`)

Function	Returns	Description
`timeline.get_playhead()`	`number`	Current playhead position in seconds.
`timeline.set_playhead(t)`	`void`	Moves timeline playhead to `t` seconds.
`timeline.get_duration()`	`number`	Total duration of the project timeline.
`timeline.get_track_count()`	`integer`	Total number of video and audio tracks.
`timeline.place_clip(path, track, t, dur)`	`string`	Places video, audio, or image clip on track.
`timeline.place_image(path, track, t, dur)`	`string`	Places image clip on track and returns clip ID.
`timeline.add_keyframe(id, prop, t, val)`	`void`	Adds animation keyframe (opacity, volume, pos_x, etc.).
`timeline.clear_keyframes(id, prop)`	`void`	Clears all keyframes for a property on a clip.
`timeline.select_clips(ids)`	`void`	Selects multiple clips simultaneously.
`timeline.get_selected_clips()`	`table`	Returns array of selected clip IDs.
`timeline.move_track_up(idx)`	`void`	Moves track up in stack order.
`timeline.move_track_down(idx)`	`void`	Moves track down in stack order.

Function	Returns	Description
`timeline.set_track_name(idx, name)`	`void`	Renames track at index.
`timeline.get_track_name(idx)`	`string`	Gets track name at index.

3.4. Clip Transforms & Attributes (`clip`)

Function	Returns	Description
`clip.get_position(clip_id)`	`number, number`	Gets (x, y) coordinates relative to canvas center.
`clip.set_position(clip_id, x, y)`	`void`	Sets center position coordinates.
`clip.get_scale(clip_id)`	`number`	Gets uniform scale factor (1.0 = 100%).
`clip.set_scale(clip_id, s)`	`void`	Sets scale factor.
`clip.get_opacity(clip_id)`	`number`	Gets opacity [0.0, 1.0].
`clip.set_opacity(clip_id, o)`	`void`	Sets opacity.
`clip.get_volume(clip_id)`	`number`	Gets audio volume gain multiplier [0.0, 2.0].
`clip.set_volume(clip_id, v)`	`void`	Sets audio volume gain multiplier.
`clip.get_time(clip_id)`	`number, number, number`	Gets (start_time, duration, offset).
`clip.get_transform(clip_id)`	`table`	Gets transform table `{x, y, scale, rotation}`.
`clip.set_transform(clip_id, x, y, s, r)`	`void`	Sets translation X, Y, scale, and rotation simultaneously.

Function	Returns	Description
`clip.reset_transform(clip_id)`	`void`	Resets transform to origin (x=0, y=0, scale=1.0, rotation=0).

3.5. GUI Controls (`gui`)

Function	Returns	Description
`gui.text(text)`	`void`	Renders text label.
`gui.text_colored(r, g, b, a, text)`	`void`	Renders colored RGBA text.
`gui.button(label, [callback])`	`boolean`	Renders push button and executes callback on click.
`gui.slider_float(label, val, min, max)`	`number`	Renders float slider widget.
`gui.slider_int(label, val, min, max)`	`integer`	Renders integer slider widget.
`gui.drag_float(label, val, [speed, min, max])`	`number`	Renders drag float box.
`gui.input_text(label, text)`	`string`	Renders single-line text input field.
`gui.checkbox(label, state)`	`boolean`	Renders toggle checkbox.
`gui.same_line()`	`void`	Aligns next widget on the same horizontal line.
`gui.separator()`	`void`	Draws horizontal separator line.
`gui.file_dialog(title, cb, [filter])`	`void`	Opens file dialog matching user settings (Native/Built-in).
`gui.native_file_dialog(title, cb, [filter])`	`void`	Forces native Zenity/KDialog system file dialog.

Function	Returns	Description
`gui.popout_preview(popped)`	`void`	Detaches (true) or docks (false) the video preview panel into a floating window.
`gui.is_preview_popped_out()`	`boolean`	Returns whether preview is currently detached/floating.
`gui.set_preview_aspect_mode(mode)`	`void`	Sets aspect mode: 0 (Auto), 1 (16:9), 2 (9:16), 3 (4:3), 4 (1:1), 5 (21:9), 6 (4:5).
`gui.get_preview_aspect_mode()`	`integer`	Returns current preview aspect mode index (0-6).
`gui.set_transform_tool(tool)`	`void`	Sets active preview gizmo tool ("select", "translate", "scale", "rotate").
`gui.get_transform_tool()`	`string`	Returns active preview gizmo tool name.

3.6. Engine Utilities (`minicut`)

Function	Returns	Description
`minicut.push_undo()`	`void`	Records snapshot for Undo history.
`minicut.set_status(msg, [dur])`	`void`	Displays temporary toast status message.
`minicut.log(msg)`	`void`	Outputs debug log message to console.

3.7. Export Automation & Progress (`export`)

Function	Returns	Description
`export.get_audio_progress()`	`number`	Audio mixing progress (stereo mixing & keyframes, 0.0 to 1.0).
`export.get_progress()`	`number`	Overall export pipeline progress (0.0 to 1.0).

Function	Returns	Description
`export.is_exporting()`	`boolean`	True if background render/encode is actively running.
`export.get_status()`	`string`	Human-readable pipeline status description.